

Construcción y análisis de una función hash artesanal

Tarea de laboratorio • BlockChain

Universidad Anáhuac México • Facultad de Ingeniería

José de Jesús Ángel Ángel

Objetivo general

Diseñar, implementar y **atacar experimentalmente** una función hash propia, con el fin de comprender de manera cuantitativa las propiedades de seguridad fundamentales: *efecto avalancha*, *resistencia a colisiones*, *resistencia a preimagen* y *resistencia a segunda preimagen*. El énfasis de la tarea no está en construir una función segura, sino en **medir con rigor por qué no lo es**.

Advertencia académica

La función construida en esta práctica es un objeto **didáctico**. Bajo ninguna circunstancia debe emplearse en un sistema real. Toda aplicación productiva debe usar primitivas estandarizadas (SHA-2, SHA-3, BLAKE2/3).

Parte 1. Construcción de la función hash

Cada equipo debe construir una función

$$\mathcal{H} : \{0, 1\}^* \longrightarrow \{0, 1\}^n,$$

de manera *artesanal*, es decir, sin invocar ninguna primitiva criptográfica existente (no se permite usar `hashlib`, MD5, SHA-*, CRC, ni bibliotecas equivalentes).

Requisitos mínimos de diseño:

- 1.1 Longitud de salida.** Elegir $n \in \{16, 24, 32\}$ bits (equivalentemente 4, 6 u 8 dígitos hexadecimales) y justificar la elección. Una salida corta es indispensable para que los ataques de las Partes 3 y 4 sean computacionalmente viables.
- 1.2 Entrada de longitud arbitraria.** Definir un esquema de relleno (*padding*) que garantice que el mensaje sea múltiplo del tamaño de bloque y que codifique la longitud original (estilo Merkle-Damgård).
- 1.3 Estructura iterativa.** Definir un estado interno h_0 (vector de inicialización) y una función de compresión f tal que

$$h_i = f(h_{i-1}, m_i), \quad i = 1, \dots, t, \quad \mathcal{H}(M) = h_t.$$

- 1.4 Operaciones sugeridas.** Al menos tres familias distintas entre: XOR, suma modular, rotaciones (*ROTL/ROTR*), desplazamientos, multiplicación por constantes impares, sustitución mediante una S-Box propia, permutación de bytes.

- 1.5 Número de rondas.** Parametrizable. Se pedirá comparar el comportamiento con pocas y con muchas rondas.
- 1.6 Determinismo y eficiencia.** La misma entrada debe producir siempre la misma salida, y el cómputo debe ser suficientemente rápido para permitir millones de evaluaciones.

Esqueleto sugerido (pseudocódigo)

```

1 funcion H(M, n, rondas):
2   M <- padding(M) # bloques de 32 bits + longitud
3   h <- IV # constante de 32 bits, no trivial
4   para cada bloque m en M:
5     para r de 1 hasta rondas:
6       h <- (h + m) mod 2^32 # difusion aritmetica
7       h <- ROTL(h, 7) # difusion posicional
8       h <- h XOR SBOX(h) # no linealidad
9       h <- (h * C) mod 2^32 # C constante impar
10    fin
11  fin
12  devolver truncar(h, n) # n bits de salida

```

Parte 2. Medición del efecto avalancha

Criterio estricto de avalancha (SAC)

Una función hash ideal satisface que, al cambiar **un solo bit** de la entrada, cada bit de la salida cambia con probabilidad $1/2$. Es decir, la distancia de Hamming esperada entre los dos digestos es $n/2$, lo que corresponde a un **50 %** de bits modificados.

Protocolo experimental:

- 2.1** Seleccionar un conjunto de N mensajes originales $\{M_1, \dots, M_N\}$ de distinta naturaleza (texto en español, cadenas aleatorias, mensajes muy cortos, mensajes largos, mensajes con alta repetición). Se recomienda $N \geq 30$.
- 2.2** Para cada mensaje M_j de L_j bits, generar todas (o una muestra aleatoria de K) las variantes $M_j^{(i)}$ que difieren exactamente en el bit i .
- 2.3** Calcular la distancia de Hamming entre digestos:

$$\delta_{j,i} = d_H\left(\mathcal{H}(M_j), \mathcal{H}(M_j^{(i)})\right).$$

- 2.4** Reportar el porcentaje promedio de bits modificados:

$$AV = \frac{1}{N K} \sum_{j=1}^N \sum_{i=1}^K \frac{\delta_{j,i}}{n} \times 100 \%.$$

- 2.5** Reportar además la desviación estándar, el mínimo y el máximo de $\delta_{j,i}$, y el **histograma** de la distribución de distancias de Hamming (debe aproximarse a una binomial $\text{Bin}(n, 1/2)$).

2.6 Repetir el experimento variando el número de rondas y graficar AV como función de las rondas.

Debe reportarse explícitamente

- El número N de **mensajes originales** utilizados.
- El número K de **intentos** (bits volteados) por mensaje.
- El total de evaluaciones $N \times K$ de la función hash.
- El porcentaje AV y su desviación estándar.

Formato de la tabla de resultados sugerida:

Mensaje	Long.	Intentos	$\bar{\delta}$ (bits)	% cambiado	σ	[mín, máx]
M_1 (texto)	128 b	128				
M_2 (aleatorio)	256 b	256				
M_3 (repetitivo)	64 b	64				
$\vdots$						
Global	—	$N \times K$				

Interpretación exigida: si AV se aleja significativamente de 50 %, explicar *qué componente del diseño* lo provoca (difusión insuficiente, linealidad, rotaciones mal elegidas, pocas rondas).

Parte 3. Búsqueda de colisiones

Fundamento: paradoja del cumpleaños

Para una salida de n bits, el número esperado de evaluaciones antes de encontrar una colisión es

$$E[Q] \approx \sqrt{\frac{\pi}{2}} \cdot 2^{n/2} \approx 1.2533 \cdot 2^{n/2},$$

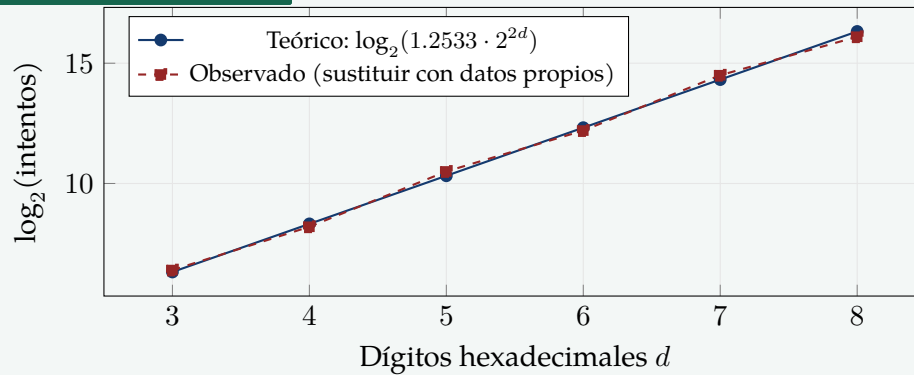
frente a 2^n de una búsqueda exhaustiva. Si se trunca el digesto a d dígitos hexadecimales, entonces $n = 4d$ y $E[Q] \approx 1.2533 \cdot 2^{2d}$.

Protocolo experimental:

- 3.1 Truncar $\mathcal{H}$ a los **primeros** $d = 3$ **dígitos hexadecimales** (12 bits) y buscar dos mensajes distintos $M \neq M'$ con $\mathcal{H}_3(M) = \mathcal{H}_3(M')$.
- 3.2 Implementar la búsqueda con una tabla *hash* (diccionario) de digesto $\rightarrow$ mensaje, generando mensajes aleatorios hasta la primera repetición.
- 3.3 **Repetir el experimento al menos 100 veces** con semillas distintas y reportar: número medio de intentos, mediana, desviación estándar y **tiempo medio** en milisegundos.
- 3.4 Incrementar el truncamiento a $d = 4, 5, 6, 7$ (y $d = 8$ si el hardware lo permite) repitiendo el proceso, y reportar los tiempos en cada caso.
- 3.5 Graficar $\log_2(\text{tiempo})$ contra d y verificar empíricamente la pendiente teórica esperada de 2 (es decir, el tiempo se multiplica por ≈ 4 por cada dígito hexadecimal añadido).
- 3.6 Comparar el número de intentos observado contra el valor teórico $1.2533 \cdot 2^{2d}$ y calcular el error relativo.

d (hex)	n (bits)	Intentos teóricos	Intentos obs.	Tiempo medio	Repeticiones	Error rel.
3	12	≈ 80			100	
4	16	≈ 321			100	
5	20	$\approx 1\,283$			50	
6	24	$\approx 5\,133$			50	
7	28	$\approx 20\,530$			20	
8	32	$\approx 82\,120$			10	

Plantilla de gráfica (pgfplots)



Parte 4. Búsqueda de preimágenes (imágenes inversas)

Fundamento: costo de la preimagen

Dado un objetivo $y \in \{0, 1\}^n$, encontrar M tal que $\mathcal{H}(M) = y$ requiere, para una función ideal, un número de intentos que sigue una distribución geométrica de parámetro $p = 2^{-n}$, con esperanza

$$E[Q] = 2^n = 2^{4d}.$$

Nótese el contraste fundamental: la preimagen es **cuadráticamente más cara** que la colisión.

Protocolo experimental:

- 4.1 Fijar un objetivo y truncado a $d = 3$ dígitos hexadecimales. Se sugiere usar como objetivo el hash del nombre del equipo.
- 4.2 Buscar por fuerza bruta mensajes aleatorios hasta hallar $\mathcal{H}_3(M) = y$. Repetir al menos 100 veces con objetivos distintos.
- 4.3 Reportar intentos medios, tiempo medio y desviación estándar.
- 4.4 Incrementar $d = 4, 5, 6$ (y $d = 7$ solo si el tiempo estimado es razonable) y reportar los tiempos.
- 4.5 Graficar $\log_2(\text{tiempo})$ contra d ; la pendiente esperada ahora es 4 (el tiempo se multiplica por ≈ 16 por cada dígito hexadecimal añadido).
- 4.6 **Comparación explícita:** construir una gráfica que superponga las curvas de colisión y de preimagen, y discutir la separación cuadrática entre ambas. Esta comparación se ampliará en la síntesis final con la curva de segunda preimagen.

d (hex)	n (bits)	Intentos teóricos	Intentos obs.	Tiempo medio	Repeticiones
3	12	4 096			100
4	16	65 536			100
5	20	1 048 576			30
6	24	16 777 216			10
7	28	268 435 456			3

Verificación de la separación teórica

Si la implementación es correcta, para un mismo d debe observarse

$$\frac{T_{\text{preimagen}}(d)}{T_{\text{colisión}}(d)} \approx \frac{2^{4d}}{1.2533 \cdot 2^{2d}} = \frac{2^{2d}}{1.2533}.$$

Para $d = 4$ esto equivale a un factor cercano a 200. Si el cociente observado es mucho menor, la función tiene una **debilidad estructural** que debe investigarse y reportarse.

Parte 5. Resistencia a segunda preimagen**Las tres nociones, formalmente**

Sea $\mathcal{H} : \{0,1\}^* \rightarrow \{0,1\}^n$. Las tres nociones de seguridad se distinguen **por quién elige los mensajes**:

- **Colisión.** Hallar *cualquier* par $M \neq M'$ con $\mathcal{H}(M) = \mathcal{H}(M')$. El atacante elige *ambos*. Costo ideal: $\mathcal{O}(2^{n/2})$.
- **Preimagen.** Dado y , hallar M con $\mathcal{H}(M) = y$. El atacante *no* elige nada. Costo ideal: $\mathcal{O}(2^n)$.
- **Segunda preimagen.** Dado M fijo, hallar $M' \neq M$ con $\mathcal{H}(M') = \mathcal{H}(M)$. El atacante elige *solo uno* de los dos mensajes. Costo ideal: $\mathcal{O}(2^n)$.

La segunda preimagen es el escenario realista de la falsificación documental: el atacante **no** puede modificar el contrato ya firmado, solo puede fabricar el sustituto.

El error conceptual que esta práctica debe erradicar

Es muy frecuente confundir colisión con segunda preimagen. La diferencia no es semántica sino **cuadrática**: con $d = 6$ dígitos hexadecimales ($n = 24$), una colisión cuesta unos 5 000 intentos y una segunda preimagen unos 16 700 000. El experimento debe hacer visible esa brecha de más de tres órdenes de magnitud *sobre la misma función y el mismo hardware*.

Protocolo experimental:

- 5.1 Tomar $N \geq 20$ mensajes objetivo M_j **fijos** (reutilizar el conjunto de la Parte 2 para poder correlacionar resultados). Registrar $y_j = \mathcal{H}_d(M_j)$.
- 5.2 Con $d = 3$ dígitos hexadecimales, generar mensajes aleatorios M' hasta hallar $\mathcal{H}_3(M') = y_j$ con $M' \neq M_j$. Repetir al menos 100 veces con semillas distintas.
- 5.3 Reportar intentos medios, mediana, desviación estándar y **tiempo medio**.
- 5.4 Barrer $d = 4, 5, 6$ (y $d = 7$ si el hardware lo permite) y reportar los tiempos. La pendiente esperada de $\log_2(\text{tiempo})$ contra d es nuevamente 4.
- 5.5 **Contraste con la Parte 4.** Calcular el cociente

$$\rho(d) = \frac{T_{2^{\text{a preimagen}}}(d)}{T_{\text{preimagen}}(d)}.$$

Teóricamente $\rho(d) \approx 1$, porque ambos ataques tienen el mismo costo esperado 2^{4d} . Verificar estadísticamente que la diferencia observada no es significativa (basta comparar las medias con sus intervalos de confianza).

5.6 Contraste con la Parte 3. Calcular

$$\frac{T_{2^{\text{a}}\text{preimagen}}(d)}{T_{\text{colisión}}(d)} \approx \frac{2^{2d}}{1.2533},$$

y discutir por qué el atacante que *puede* elegir los dos mensajes es cuadráticamente más poderoso.

d (hex)	n (bits)	Intentos teóricos	Intentos obs.	Tiempo medio	Repeticiones	$\rho(d)$
3	12	4 096			100	
4	16	65 536			100	
5	20	1 048 576			30	
6	24	16 777 216			10	
7	28	268 435 456			3	

5.7 Búsqueda de segundas preimágenes estructurales. La fuerza bruta anterior es el escenario ideal. Ahora debe intentarse obtener segundas preimágenes **sin costo**, explotando defectos del diseño. Aplicar sistemáticamente las siguientes pruebas y reportar el resultado de cada una:

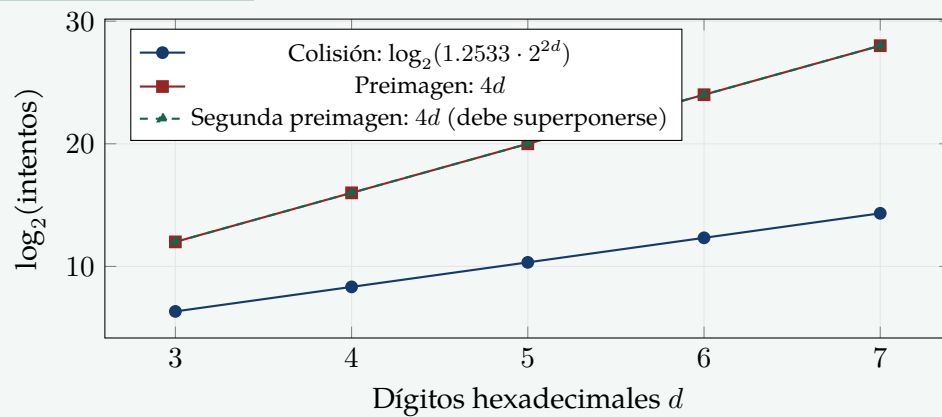
Prueba	Qué se explota	¿Funciona?
Relleno ambiguo	$\mathcal{H}(M) \stackrel{?}{=} \mathcal{H}(M \parallel 0x00 \dots)$. Si el <i>padding</i> no codifica la longitud, se obtiene una segunda preimagen gratuita.	
Permutación de bloques	$\mathcal{H}(m_1 \parallel m_2) \stackrel{?}{=} \mathcal{H}(m_2 \parallel m_1)$. Falla si f es conmutativa (típico si solo se usa XOR o suma).	
Extensión de longitud	Dado $\mathcal{H}(M)$ sin conocer M , calcular $\mathcal{H}(M \parallel X)$. Debilidad clásica de Merkle–Damgård sin <i>finalización</i> .	
Linealidad sobre $\mathbb{F}_2$	Buscar Δ tal que $\mathcal{H}(M \oplus \Delta) = \mathcal{H}(M)$ para <i>todo</i> M : un solo Δ rompe la función por completo.	
Puntos fijos	Hallar (h, m) con $f(h, m) = h$: permite insertar bloques sin alterar el digesto.	

Síntesis comparativa de los tres ataques

Ataque	El atacante elige	Costo ideal	Pendiente en d	Intentos ($d = 6$)
Colisión	M y M'	$1.2533 \cdot 2^{n/2}$	2	$\approx 5\,133$
Segunda preimagen	solo M'	2^n	4	$\approx 1.68 \times 10^7$
Preimagen	nada	2^n	4	$\approx 1.68 \times 10^7$

Debe incluirse una **única gráfica final** con las tres curvas empíricas superpuestas sobre las teóricas. Las curvas de preimagen y de segunda preimagen deben *coincidir*; si no lo hacen, hay un sesgo en la implementación o en el muestreo que debe explicarse.

Plantilla de la gráfica final



Reporte del entorno de cómputo

Todos los tiempos reportados carecen de sentido sin la especificación del hardware. Debe incluirse la siguiente ficha:

Ficha técnica del entorno	
Procesador (CPU)	modelo, frecuencia base/turbo, núcleos/hilos
Memoria RAM	capacidad y tipo
GPU (si se usó)	modelo y VRAM
Sistema operativo	nombre y versión
Lenguaje / versión	p. ej. Python 3.12, C++20, Rust 1.7x
Compilador / flags	p. ej. -O3, JIT, sin optimización
Paralelismo	un hilo / k hilos / vectorizado
Método de medición	reloj monótono, número de repeticiones
Hash/segundo (<i>throughput</i>)	evaluaciones por segundo medidas

Rúbrica analítica de evaluación

Construcción y análisis de una función hash artesanal

Criterio	4 — Sobresaliente	3 — Competente	2 — En desarrollo	1 — Insuficiente	Nivel
1. Diseño e implementación de la función <i>Peso: 15 %</i>	Construcción original y determinista; <i>padding</i> que codifica la longitud; tres o más familias de operaciones, cada una justificada por la propiedad que aporta (difusión, no linealidad, asimetría); rondas parametrizables; diagrama formal de f ; batería de pruebas.	Función correcta y funcional, con <i>padding</i> válido y operaciones adecuadas, pero justificadas de manera superficial; diagrama presente aunque incompleto.	La función opera, pero con defectos: <i>padding</i> sin codificar longitud, una sola familia de operaciones o número de rondas fijo; documentación parcial del diseño.	No es determinista, no ejecuta, o se apoya en una primitiva criptográfica existente (<code>hashlib</code> , <code>CRC</code> , etc.).	
2. Diseño y ejecución experimental <i>Peso: 15 %</i>	Cumple o excede N , K y las réplicas en las cuatro prácticas; barridos completos en d hasta el máximo factible; semillas controladas; <i>warm-up</i> descartado; reloj monótono; <i>throughput</i> medido por separado.	Cumple los mínimos en las cuatro prácticas, con barridos completos y medición correcta; alguna serie con menos réplicas de las pedidas.	Falta una práctica, o los barridos se truncan sin justificación; muestras pequeñas; mediciones sin réplicas ni control de semilla.	Resultados anecdóticos (una sola corrida), o faltan dos o más prácticas.	
3. Matemáticas empleadas <i>Peso: 20 %</i>	Uso correcto y explícito de la distancia de Hamming y del modelo $\text{Bin}(n, 1/2)$; aproximación del cumpleaños con la constante $\sqrt{\pi/2}$; distribución geométrica de media 2^n ; notación formal de $\mathcal{H}$, f y de las tres nociones; estadística inferencial (media, mediana, σ , intervalos de confianza); pendientes en escala $\log_2$ y errores relativos correctamente derivados.	Modelos teóricos bien aplicados y estadísticos descriptivos completos; imprecisiones menores de notación o ausencia de intervalos de confianza.	Las fórmulas se citan pero se aplican mecánicamente; se reporta solo la media; aparece alguna confusión entre los órdenes $2^{n/2}$ y 2^n ; las pendientes se afirman sin calcularse.	Ausencia de modelo teórico: los datos se presentan sin contraste matemático, o hay errores conceptuales que invalidan el análisis.	
4. Razonamiento y conclusiones <i>Peso: 25 %</i>	Cada resultado se contrasta con su predicción teórica, la discrepancia se cuantifica y se atribuye a un componente concreto del diseño; explica la jerarquía colisión $\ll$ 2ª preimagen $\approx$ preimagen en términos del poder del atacante; reconoce los límites de validez (tamaño de muestra, hardware, n pequeño); la conclusión crítica se sostiene íntegramente en los datos propios.	Interpreta correctamente los resultados principales y los contrasta con la teoría; las explicaciones causales son plausibles pero poco específicas; los límites de validez se mencionan brevemente.	Se describen los resultados más de lo que se interpretan; conclusiones genéricas sin evidencia que las respalde; no se distinguen con claridad las tres nociones de seguridad.	No hay conclusiones, o las conclusiones son contradichas por los datos que el propio equipo reporta.	

Criterio	4	3	2	1	Nivel
5. Escritura y completitud del reporte <i>Peso: 15 %</i>	Estructura completa (resumen, diseño, método, resultados, discusión, conclusión, referencias); todas las tablas y gráficas llenas, numeradas, con unidades y referidas en el texto; ficha del entorno de cómputo completa; prosa técnica precisa y sin errores; extensión respetada.	Estructura completa y contenido presente; alguna tabla sin unidades o figura no referida en el texto; redacción correcta con descuidos menores.	Faltan secciones o hay tablas incompletas; figuras sin leyenda; redacción telegráfica o confusa; se excede la extensión sin justificación.	Reporte fragmentario, notas sin redactar, o entrega limitada al código y a capturas de pantalla.	
6. Reproducibilidad y entrega técnica <i>Peso: 10 %</i>	Repositorio con instrucciones de ejecución, semillas fijadas y documentadas, datos crudos en CSV y scripts que regeneran todas las tablas y gráficas del reporte; la ejecución se verifica en una máquina distinta.	Código, datos crudos e instrucciones entregados; semillas documentadas; las tablas se regeneran de forma manual.	Código sin instrucciones o sin datos crudos; semillas no documentadas; resultados no reproducibles a partir de lo entregado.	No se entrega el código, o el código entregado no ejecuta.	